Peak Powerhouse Database Management System

Team: Peak Data (Group 3)

Arthur Adjamoglian and Min Deng

Feedback by the TAs and peer reviewer

Feedback from Pavel Gorshenkov

This database back-end implementation is perfect introduction into transactional DBMS concepts of entities, attributes, constraints and relationships between these characteristics. Designing a system for a gym business is also a great way to conceptualize these tools and to put them into use. This business has a clear need for managing a database with three locations and a unique set of employees, equipment, members, and transactions. You have clearly outlined the gym-related services and a general overview of information storing and management.

In terms of raw numbers, you specified a gym-member count, number of gym locations and monthly transactions. Transactions that include monthly membership plans, training sessions, and other gym-related items. Good design choices were made in centralizing the gym location entity which then shares a location foreign key id with employees, sales, members and equipment. This is effectively visualized in the ERD.

Entities Listed: Locations, Equipment, Roles, Employees, Memberships, Members, Items, Sales, sales_has_items, Sessions.

It is worth mentioning, that sales_has_items should be labeled as an associative/relationship table as it is meant to facilitate a many-many relationship between items and sales.

Roles: This is a wise design choice as a role change or update would result in a change in one row instead of sorting through every employee table updating each role string.

The Employees table could be populated with more attributes consistent with a modern workplace title. In most positions of employment it is common to include a residence address, email address, phone number, date of employment, etc.

In the ERD and in the database design it would make more sense to have a direct, one-to-many relationship between 'sales_has_items' and 'sessions'. Does bridging the gap between 'sales_has_items' and 'sessions' feel less structurally awkward from a structural business point of view?

The outline correctly formats and presents a well defined 1:M relationship under each entity. At least one M:M relationship is also present between sales and items. It might be worth mentioning a M:M relationship between members and sessions that may exists that isn't defined.

It's important to note that the sales entity has a transaction id and member id, quantity, etc. There is no way to differentiate between a session and other gym items. A null-able session id should be included in the sales table and a quantity attribute should only exist in sales_has_items to make a distinction between which sale of what gym product is made.

Naming conventions are sensible and consistent with plural entities and singular attributes.

All feedback is student-generated and human-authored.

Feedback from Ezri Martinez

Does the overview describe what problem is to be solved by a website with DB back end? If yes, summarize. If not, what changes would better support describing the problem to be solved?

Yes, the problem is stated as information management of various aspects of the business. Although, I think it would be helpful to be more specific on something that requires information management rather than generalizing information management as a whole. This could limit the potential for over engineering.

Does the overview list specific facts? If yes, summarize what the facts illustrate about the proposed DB solution. If not, what facts would better support illustrating the scope and scale of the proposed DB solution?

Yes, it lists that approximately 1000 sale transactions are made per month, there are approximately 2000 members, and there are three gym locations. This defines the need for an information management system by providing the scale of business operations. These facts also signify the need for a central information management service due to multiple locations.

Are at least four entities described, and does each one represent a single idea to be stored as a list? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

Yes, there are 10 entities and all represent a single idea. The relations are: Locations tracks information of each gym locations, Equipment tracks information about equipment availability, Roles tracks employment roles, Employees tracks employee statistics, Memberships track membership plan information, Members tracks member statistics, Items tracks items/services availability, Sales tracks sale transactions, sale_has_items connects Sales and Items, and Sessions tracks items linked to scheduled sessions.

Does the outline of entity details describe the purpose of each, list attribute datatypes and constraints, and describe relationships between entities? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

Yes, the entity details have a purpose, datatypes, constraints and describe the relationships between entities. The list of entities has fields for each of these as a description, datatypes, and relationships. However, I noticed some datatypes might be better suited as another datatype. For example, member_phone could be varchar rather than int. Also I think that any currency cost attributes should be decimal rather than int.

Are 1:M relationships correctly formulated? Is there at least one M:M relationship? Does the ERD present a logical view of the database? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

Yes, the 1:M relationships are correct and there is one M:M relationship. The 1:M relationships are as follows: locations to equipment, employees, members, and sales, roles to employees, and items to sessions. The M:M relation is between sales and items through sales_has_items with a PK. The ERD makes sense and matches the outline.

Is there consistency in a) naming between overview and entity/attributes b) entities plural, attributes singular c) use of capitalization for naming? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

a) Yes, the namings in the overview align with the entities.

b) Yes, entities are plural and attributes are singular.

c) No, sales_has_items is lowercase while the rest of the entities are capitalized. To fix this, I'd suggest making all entities lowercase to match the capitalization of the ERD.

Originality: The response to the questions above are largely my own. Claude was used to review clarity, flow, and grammar.

Anthropic. (October 12, 2026). Claude (Opus 5.5) [Large language model]. https://claude.ai.

Scope: Used to review my draft answers for clarity, grammar, and writing flow.

Prompts: I asked Claude to review my responses to questions, correct errors, and clarify missing elements.

Feedback from Jacob Gervais

Does the overview describe what problem is to be solved by a website with DB back end? If yes, summarize. If not, what changes would better support describing the problem to be solved?

Your introductory paragraph does explicitly describe what problem a database will be solving: "The system needs to manage information about members, employees, equipment, membership plans, training sessions, and sales. A database-backed website will help the business organize this information and keep track of the relationships between members, employees, locations, memberships, and sales."

I think one or two sentences describing the current hardship or main difficulties prior to having a DB backend would make this introductory paragraph stronger.

Additionally, while one paragraph could be sufficient, splitting your paragraph into 2 smaller (and more focused) paragraphs could be a good idea to improve readability. One paragraph describing the business and the scale of transactions per year, members, etc., and another paragraph describing the main problem(s).

Does the overview list specific facts? If yes, summarize what the facts illustrate about the proposed DB solution. If not, what facts would better support illustrating the scope and scale of the proposed DB solution?

The overview does list a few specific facts. I see that there are 2000 members and 1000 transactions per month. This information is good but I cant help but feel there should be more. For instance, there should be a mention of membership cancellations and renewals maybe.

In total, these facts illustrate for me a small-to-medium scale database would be necessary to handle the day-to-day transactions at this gym. With 3 locations, the database will need to be hosted online in order to synchronize data between the 3 locations.

Are at least four entities described, and does each one represent a single idea to be stored as a list? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

There are 10 entities described and each one represents a single listable object / event.

There is Locations, Equipment, Roles, Employees, Memberships, Members, Items, Sales, sales_has_items, and Sessions.

These are all represent a single idea stored as a list and are logically named for the most part. I am just a little confused by a couple things.

Why is “sales_has_items” all lowercase?

What is “Sessions”? Even after reading the description, I really am not sure what it is supposed to be representing.

For readability, I would consider adding more detail to your description of each entity if they are not super straight-forward.

Does the outline of entity details describe the purpose of each, list attribute datatypes and constraints, and describe relationships between entities? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

The purpose of each entity is described, although fairly briefly. I think one additional sentence would be helpful for a few of them.

Each entity does list attributes, datatypes, and constraints. The only thing I would argue with here is that anything money related probably shouldn’t be an INT (e.g. membership_monthly_charge should probably be decimal (2,2))

The relationships between entities are very well described. This helps illustrate the flow of the database. No additional writing is required here.

Are 1:M relationships correctly formulated? Is there at least one M:M relationship? Does the ERD present a logical view of the database? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

The relationships described and on the diagram appear to be correctly formulated.

There is a M:N relationship between sales and items. This relationship is facilitated by the sales_has_items intersection table.

The ERD looks great with all relationships looking proper as described above. All attributes are visible and there are symbols representing each attribute type which helps with understanding the ERD.

Is there consistency in a) naming between overview and entity/attributes b) entities plural, attributes singular c) use of capitalization for naming? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

The naming between the overview and the ERD is consistent. I believe it was generated automatically using MySQL workbench.

The entities are all plural, while the attributes are all singular.

The entities are all lowercase in the ERD. They should begin with a capital. The attributes use snake casing and are all singular.

Overall, the ERD looks great and should not take too long to fix the capitalization issue.

As a reviewer, clearly describe to what extent your feedback to the team was original (e.g. "all my work") or non-original (e.g. used AI tools per Code Citation Tips).

This review was done using zero AI or outside resources whatsoever.

Feedback from Thien Ngan Nguyen

The overview clearly explains why Peak Powerhouse would benefit from a database backed website. The business has three locations and needs to keep information about members, employees, equipment, memberships, sessions and sales together in one system. The facts about having around 2000 members and 1000 sales transactions each month also help show the size of the database and why keeping the information organized is important. There are

more than four entities and the main entities each represent a clear type of information that needs to be stored, such as Locations, Employees, Members, Items and Sales. The outline also explains the purpose of the entities and includes attributes, datatypes, constraints and relationships. One thing I noticed is that manager_id is included in Locations but it is not marked as a foreign key or explained as a relationship so I think that connection should be clarified. Some money related attributes such as price and membership charges may also work better as DECIMAL instead of INT. The 1:M relationships generally make sense and there is an M:N relationship between Sales and Items that is resolved using sales_has_items. I would double check the key for sales_has_items, though because only sales_transaction_id appears to be marked as the primary key. Since one sale can contain several different items, I think both foreign keys could form a composite primary key. I would also move quantity from Sales into sales_has_items since each item in the same sale could have a different quantity. The ERD otherwise matches the written outline and gives a logical view of the database. Naming is mostly consistent with plural entity names and singular attributes, although sales_has_items uses a different capitalization style from the other entity names. Overall, I think the design is easy to follow and the main improvements I would suggest are clarifying the manager relationship and checking the structure of the Sales to Items intersection table.

Originality: I reviewed the project draft myself and used Grammarly to check my spelling

Feedback from Razeel Hassan Ibrahim Rasheed(TA)

Good work on your proposal! Your overview is clear, and you've included enough numerical examples. Your database outline and ERD look good overall. One recommendation is to simplify the scope of your project, since you currently have 10 tables, which could be difficult to manage within the term. Around 5–7 tables should be enough for this project, so consider focusing on the most important features. Also, since a sale can contain multiple items, consider moving quantity to sales_has_items. Finally, clarify whether manager_id in Locations refers to an employee, since that relationship isn't shown in the ERD. Just a reminder that the overall project requires a Python class as the application interface to your database, rather than a website.

Actions Based on the Feedback

After reviewing the feedback from our four classmates and the TA, we discussed their suggestions and made several changes to improve our database design.

1. Updated Data Types (Ezri, Jacob, Thien)

We changed money-related attributes from INT to DECIMAL, including product prices, product costs, and membership monthly charges. We also changed phone number attributes from INT to VARCHAR(10) because our gym operates in the United States and we plan to store phone numbers as 10-digit numbers without formatting.

2. Improved the Sales Structure (Pavel, Thien, TA)

We renamed sales_has_items to sales_details and moved quantity from Sales to Sales Details. This allows each product in a transaction to have its own quantity. We also added sales_details_id as the primary key for the Sales Details table.

3. Simplified the Database Structure (TA)

Based on the TA's suggestion, we reduced our database from 10 tables to 7 tables to make the project more manageable. We removed Equipment, Roles, and Personal_training_sessions to focus on the main features of our gym business.

Instead of having a separate Roles table, we added employee_role to Employees to store employee job titles. We also decided to include personal training services in Products, so members can still purchase training sessions without having a separate table for scheduling appointments.

4. Added More Employee Information (Pavel)

Based on Pavel's suggestion, we added employee_phone, employee_email, and employee_address to the Employees table to store more useful employee information.

5. Simplified the Location Manager Relationship (Thien, TA)

Thien and the TA suggested clarifying the manager_id attribute in Locations. After reviewing our database design, we decided to remove this attribute to keep the structure simpler. Since the Employees table already includes employee_role and locations_location_id, we can still identify employees' job roles and their assigned gym locations without creating an additional manager relationship.

6. Improved the Project Overview (Ezri, Jacob, TA)

We rewrote the Project Overview to explain our gym's business model and operations more clearly. We added details about monthly memberships, personal training services, retail products, employees, and sales transactions. We also described the challenges of managing information across three locations and how a centralized database could help solve these problems. Following the TA's instructions, we removed the reference to a database-backed website.

Suggestions We Decided Not to Follow

1. Entity Naming Consistency (Ezri, Jacob, Thien)

Some reviewers suggested changing the capitalization of our entity names. After discussing this, we decided to keep all entity names lowercase in our ERD. We think lowercase names are easier to read and work with, especially when writing SQL queries. We also updated our entity names to follow a consistent lowercase naming convention, so this still addresses the main concern about naming consistency.

2. Adding a Direct Relationship Between Sales Details and Personal Training Sessions (Pavel)

Pavel suggested adding a direct 1:M relationship between sales_has_items (now sales_details) and Sessions.

We decided not to add this relationship because we removed the Sessions table after simplifying our database based on the TA's feedback. Personal training is now treated as a service in Products, and Sales Details records its purchase. Our database will focus on sales transactions rather than appointment scheduling.

3. Adding an M:N Relationship Between Members and Sessions (Pavel)

Pavel also suggested considering an M:N relationship between Members and Sessions.

We decided not to add this relationship because our project no longer includes a separate Sessions table. Our gym focuses on one-on-one personal training rather than group classes, and we only need to record the purchase of these services. Adding a junction table to manage session bookings would make our project more complicated than necessary.

4. Including Membership Cancellations and Renewals in the Overview (Jacob)

Jacob suggested mentioning membership cancellations and renewals to provide more specific business information in our Overview.

We decided not to include these details because our current database focuses on managing active members and their membership plans rather than membership cancellation and renewal history. We wanted to keep the Overview focused on the features included in our simplified database design.

Upgrades to the Draft Version

In addition to the changes based on peer and TA feedback, we made several improvements to our original database design.

1. Added Member Join Date

We added member_join_date (DATETIME) to the Members table so the gym can track when each member joined.

2. Added Employee First and Last Names

We added employee_fname (VARCHAR(50)) and employee_lname (VARCHAR(50)) to the Employees table to store employee names and make it easier to identify individual employees.

3. Added Sales Total

We added sale_total (DECIMAL(8,2)) to the Sales table to record the total amount of each transaction.

4. Improved the ERD Layout

After reducing our database from 10 tables to 8 tables, we rearranged the remaining tables in our ERD to make the relationships easier to see and understand. We placed related tables closer together and reduced overlapping relationship lines to improve readability.

Upgrades to the Step 2 Draft Version

1. Added roles table

We added a roles table to reduce data redundancy of multiple employees having the same role name in the employees table. The downside is we have an additional table that we now have to manage.

2. Increased length of phone field

The initial use of VARCHAR(11) could potentially lead to issues based on how phone numbers may be formatted. Thus based on Razeel's (TA) recommendation we lengthen this to VARCHAR(20) for employee_phone in the employees table, and member_phone in the members table.

Overview

Peak Powerhouse is a West Coast regional gym business operating three locations, offering 24/7 gym access to its members. Across all three locations, the business employs approximately 60–70 staff members and serves around 2,000 active members.

Peak Powerhouse operates on a monthly subscription model with one standard membership plan that provides 24/7 gym access. In addition to their memberships, members can purchase add-on services, such as 50-minute personal training sessions, and retail products, including gym merchandise, beverages, and fitness snacks. Both personal training services and retail products are recorded in the Products entity.

Peak Powerhouse handles membership information and daily purchases of products and services across its three locations. Managing information across multiple locations can be challenging without a centralized database. For example, member records may become inconsistent between locations, and employees may have difficulty tracking membership information and sales transactions.

To address these challenges, Peak Powerhouse will use a centralized database to organize and manage its business information. The system will help maintain member and employee records, manage membership plan information, track sales transactions and purchased products or services, and keep information consistent across all three locations.

Database Outline

The following entities describe the main parts of the Peak Powerhouse database, including their attributes, data types, constraints, and relationships.

Locations

The Locations entity stores information about each Peak Powerhouse gym location, including its address.

- location_id: INT, auto_increment, NOT NULL, PK
- location_address: VARCHAR(255), NOT NULL

Relationships:

- There is a 1:M relationship between Locations and Employees. One Location can have many Employees, and locations_location_id is used as a FK in Employees.
- There is a 1:M relationship between Locations and Members. One Location can have many Members, and locations_location_id is used as a FK in Members.
- There is a 1:M relationship between Locations and Sales. One Location can have many Sales transactions, and locations_location_id is used as a FK in Sales.

Employees

The Employees entity stores information about Peak Powerhouse gym employees, including their names, contact information, job roles, and assigned locations.

- employee_id: INT, PK
- employee_phone: VARCHAR(10)
- employee_email: VARCHAR(100)
- employee_address: VARCHAR(255)
- locations_location_id: INT, NOT NULL, FK
- employee_role: VARCHAR(45)
- employee_fname: VARCHAR(50)
- employee_lname: VARCHAR(50)

Relationships:

- There is a 1:M relationship between Locations and Employees. One Location can have many Employees, and locations_location_id is used as a FK in Employees.
- There is a 1:M relationship between Employees and Sales. One Employee can be associated with many Sales transactions, and employees_employee_id is used as a FK in Sales.

Memberships

The Memberships entity stores information about the monthly membership plan offered by Peak Powerhouse, including its name, description, and monthly charge.

- membership_id: INT, auto_increment, NOT NULL, PK
- membership_name: VARCHAR(45)
- membership_description: VARCHAR(255)
- membership_monthly_charge: DECIMAL(8,2)

Relationships:

- There is a 1:M relationship between Memberships and Members. One Membership Plan can have many Members, and memberships_membership_id is used as a FK in Members.

Members

The Members entity stores information about Peak Powerhouse gym members, including their personal information, contact details, membership plans, and join dates.

- member_id: INT, NOT NULL, PK
- member_fname: VARCHAR(50)
- member_lname: VARCHAR(100)
- member_address: VARCHAR(255)
- member_dob: DATE
- member_phone: VARCHAR(10)
- member_email: VARCHAR(255)
- locations_location_id: INT, NOT NULL, FK
- memberships_membership_id: INT, NOT NULL, FK
- member_join_date: DATETIME

Relationships:

- There is a 1:M relationship between Locations and Members. One Location can have many Members, and locations_location_id is used as a FK in Members.
- There is a 1:M relationship between Memberships and Members. One Membership Plan can have many Members, and memberships_membership_id is used as a FK in Members.
- There is a 1:M relationship between Members and Sales. One Member can have many Sales transactions, and members_member_id is used as a FK in Sales.

Products

The Products entity stores information about products and services available for purchase at Peak Powerhouse, including gym merchandise, beverages, fitness snacks, and personal training services.

- product_id: INT, NOT NULL, PK
- product_name: VARCHAR(100)
- product_description: VARCHAR(255)
- product_price: DECIMAL(8,2)
- product_cost: DECIMAL(8,2)

Relationships:

- There is a M:N relationship between Sales and Products, managed through the Sales_details entity. One Sale can include multiple Products, and one Product can appear in multiple Sales transactions.
- The products_product_id attribute is used as a FK in Sales_details, referencing Products.product_id.

Sales

The Sales entity stores information about sales transactions at Peak Powerhouse, including the total amount of each sale, the member making the purchase, the employee handling the transaction, and the gym location where the sale takes place.

- transaction_id: INT, NOT NULL, PK
- sale_total: DECIMAL(8,2)
- members_member_id: INT, NOT NULL, FK
- employees_employee_id: INT, NOT NULL, FK
- locations_location_id: INT, NOT NULL, FK

Relationships:

- There is a 1:M relationship between Members and Sales. One Member can have many Sales transactions, and members_member_id is used as a FK in Sales.
- There is a 1:M relationship between Employees and Sales. One Employee can handle many Sales transactions, and employees_employee_id is used as a FK in Sales.
- There is a 1:M relationship between Locations and Sales. One Location can have many Sales transactions, and locations_location_id is used as a FK in Sales.
- There is a 1:M relationship between Sales and Sales_details. One Sale can contain multiple Sales_details records, and sales_transaction_id is used as a FK in Sales_details.

Sales_details

The Sales_details entity stores information about the individual products or services included in each sales transaction, including the quantity purchased.

- sales_details_id: INT, NOT NULL, PK
- sales_transaction_id: INT, NOT NULL, FK
- products_product_id: INT, NOT NULL, FK
- quantity: INT

Relationships:

- There is a 1:M relationship between Sales and Sales_details. One Sale can have multiple Sales_details records, and sales_transaction_id is used as a FK in Sales_details.

- There is a 1:M relationship between Products and Sales_details. One Product can appear in multiple Sales_details records, and products_product_id is used as a FK in Sales_details.
- Sales_details serves as the intersection table for the M:N relationship between Sales and Products.

Roles

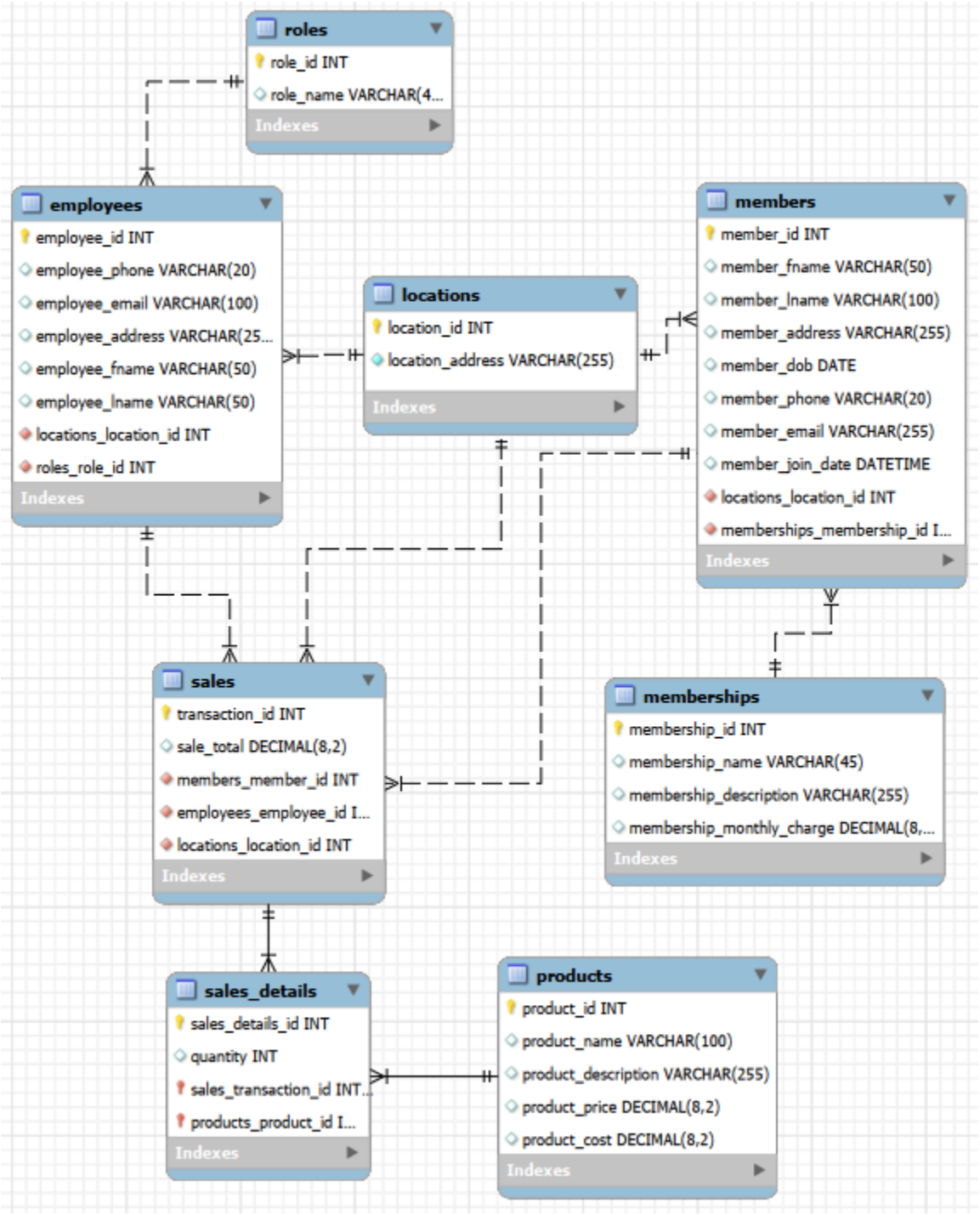
The roles entity stores information about the different roles/job titles an employee will have. The purpose of this table is to reduce the data redundancy of repeating similar roles in the employee table. Each employee will have at most one role.

- roles_role_id: INT, NOT NULL, PK
- role_name: VARCHAR(45)

Relationships:

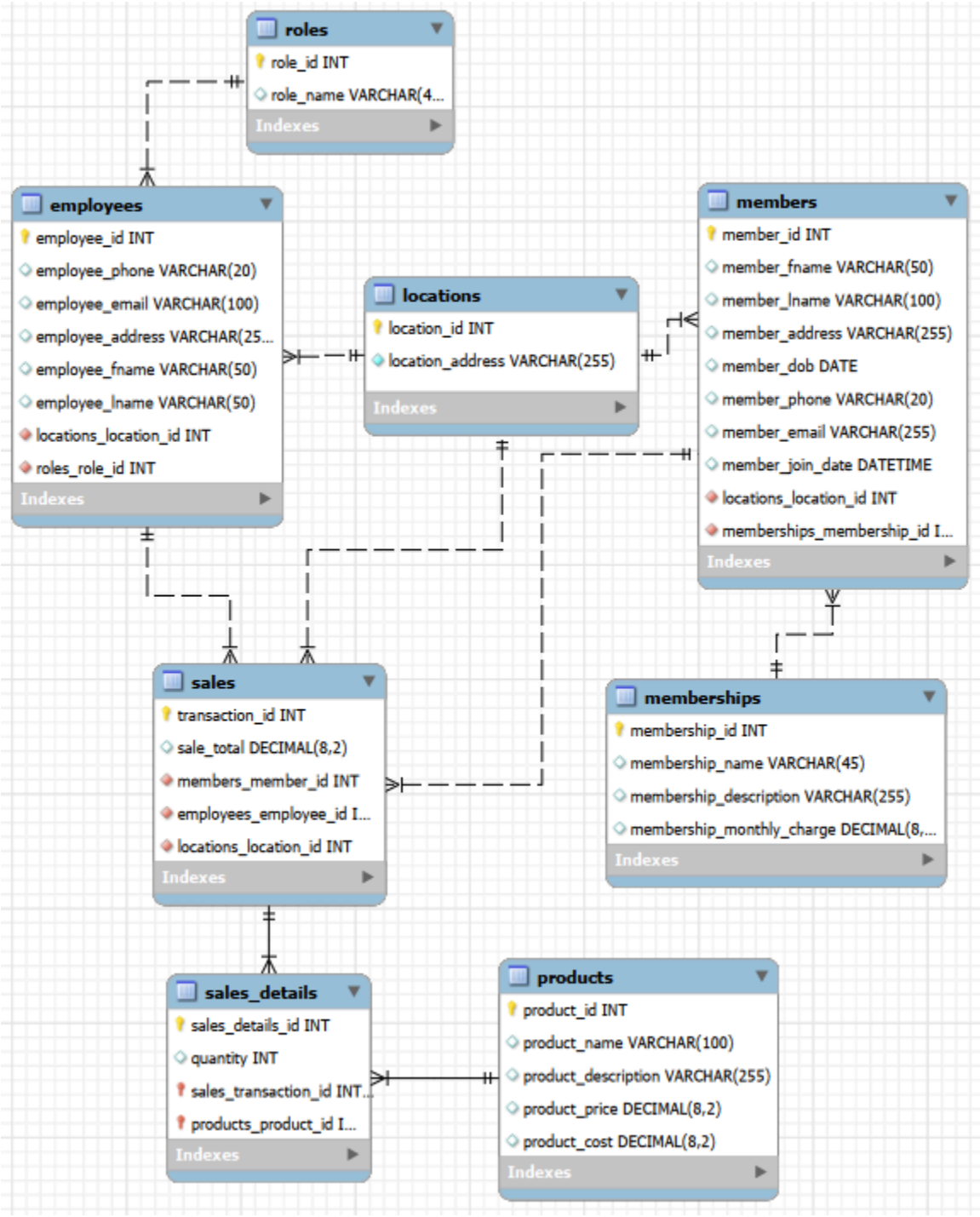
- There is a 1:M relationship between Roles and Employees. One Role can apply to multiple different employees. roles_role_id is a foreign key in the employees table.

Entity-Relationship Diagram



For Peak Powerhouse we have 7 main tables. Our gym is based upon three different sites stored in the locations table.This is then related to the employees and the members based upon which location they work/workout at. Locations is also connected to sales because where a sale occurs is also logged in out system. For a sale the employee and the member involved in that transaction are also recorded. We also have a product table that is M:M to the sales table. Memberships are obviously related to members and employees are related to roles via what position/role they have in the company.

Schema Diagram



The tables in the schema diagram meet the requirements of 1NF because all key attributes are defined. There are no repeating groups in the tables. This is the reason why we reintroduced

the roles table. Although this does increase the table count and makes managing the database more difficult it does reduce data redundancy with repeated roles in the employees table. Also, all tables were generated with a primary key. Thus, the tables are in 1NF.

To satisfy 2NF tables must be in 1NF and include no partial dependencies. The entities and all of their related attributes have been parsed out and their only dependency is to their primary key. Attributes are only related to their specific entity.

To satisfy 3NF, tables must be in 2NF and contain no transitive dependencies. For our customers, employees, and locations table we use an address attribute instead of breaking this out to a separate table with a geographic identifier and thus do not achieve 3NF. We choose to leave the addresses in the various tables and keep the structure as is.

Example Data

General Selects

location_id	location_address
1	1060 W Alameda Ave, Burbank, CA 91506
2	6150 Laurel Canyon Blvd, North Hollywood, CA 91506
3	12811 Victory Blvd, Valley Glen, CA 91506

role_id	role_name
1	Front Desk Associate
2	General Manager
3	Assistant Manager
4	Maintenance Technician
5	Janitor

employee_id	employee_phone	employee_email	employee_address	employee_fname
1	4599380912	nruggiero0@cbc.ca	08 Kings Alley	Nixie
2	6225078723	eempleton1@behance.net	8257 Weeping Birch Place	Ebony
3	7687192767	awhitcher2@guardian.co.uk	1017 Debs Court	Alie
4	3335204235	mmadill3@booking.com	836 Declaration Trail	Myrtia
5	2795868841	jbohlens4@etsy.com	5 Petterle Alley	Jolene

membership_id	membership_name	membership_description
1	Single-Club Membership	Access to only one Powerhouse Gym locati
2	Multi-Club Membership	Access to all Powerhouse Gym locations.
3	Trial Membership	Trial membership for first timers. Free

member_id	member_fname	member_lname	member_address	member_dob
1	Amalia	Laddle	6 Nobel Lane	1951-02-15
2	Codee	Oscroft	88212 Stone Corner Road	1996-08-22
3	Alain	Spurr	8 Quincy Terrace	1957-04-02
4	Katti	Antonignetti	92838 Sugar Terrace	1970-06-26
5	Bambie	Higgoe	2 Monument Junction	1987-10-14

product_id	product_name	product_description
1	Energy Drink	Super caffeinated sugary power bomb of a drink!
2	Pre-Workout Shot	Dry scooped directly into the customer's mouth.
3	Snickers Bar	You are not you when you are hungry.

transaction_id	sale_total	members_member_id	employees_employee_id	locations_location_id
1	5.50	1	1	1
2	3.30	2	1	1
3	3.30	3	1	1

sales_details_id	quantity	sales_transaction_id	products_product_id
1	1	1	1
2	1	1	2
3	1	1	3

One To Many Relationship Example

employee_id	employee_fname	employee_lname	role_name	location_addre
1	Nixie	Ruggiero	Front Desk Associate	1060 W Alameda
2	Ebony	Empleton	General Manager	1060 W Alameda
3	Alie	Whitcher	Assistant Manager	1060 W Alameda
4	Myrtia	Madill	Maintenance Technician	1060 W Alameda
5	Jolene	Bohlens	Janitor	1060 W Alameda

Many To Many Relationship Example

product_name	total_units_sold	total_revenue
Energy Drink	1	4.99
Pre-Workout Shot	1	2.99
Snickers Bar	1	2.99

**Images have been cropped to fit, some tables do not all contain all related attributes related to the table.*

Citations

The database design and documentation were created by Arthur Adjamoglian and Min Deng. We received feedback from our TA, Razeel Hassan Ibrahim Rasheed and four peer reviewers: Pavel Gorshenkov, Ezri Martinez, Jacob Gervais, and Thien Ngan Nguyen. Their suggestions helped us improve our database structure, relationships, and documentation.